

AI SCIENTIST

FASE 2 | AULA 03 -

MANIPULAÇÃO DE DADOS COM SPARK

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 9
MERCADO, CASES E TENDÊNCIAS 17
O QUE VOCÊ VIU NESTA AULA? 19
REFERÊNCIAS 20

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VEM POR AÍ?

Nos últimos anos, a forma como lidamos com dados mudou radicalmente. Volumes cada vez maiores, diferentes formatos e a necessidade de respostas quase em tempo real colocaram em xeque abordagens tradicionais de processamento. É nesse cenário que o **Apache Spark** se consolida como uma das principais ferramentas para manipulação de dados em larga escala, permitindo transformar, analisar e preparar informações de maneira distribuída, eficiente e escalável. Mas afinal, o que significa “manipular dados” quando falamos de Big Data?

Nesta aula, você será convidado(a) a avançar do conceito para a prática, explorando como o Spark viabiliza operações complexas sobre grandes conjuntos de dados com simplicidade e performance. Ao longo do conteúdo, você entenderá como ler, transformar, filtrar, agregar e enriquecer dados utilizando DataFrames e APIs modernas, preparando o terreno para análises mais sofisticadas e pipelines de dados robustos. Pense nesta etapa como um ponto de virada: a partir daqui, os dados deixam de ser apenas armazenados e passam a gerar valor real para decisões, produtos e estratégias orientadas por dados.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

HANDS ON

Antes de iniciar as atividades práticas, é fundamental garantir que o conjunto de dados utilizado no hands on esteja acessível no ambiente Databricks. Nesta aula, será utilizado um dataset público que já vem disponível nativamente no **Databricks Community Edition**, armazenado no **DBFS (Databricks File System)**. O arquivo utilizado encontra-se no seguinte caminho:

`/databricks-datasets/samples/population-vs-price/data_geo.csv`

Para validar a existência do arquivo, o primeiro passo é executar um comando simples diretamente no notebook do Databricks. Essa verificação faz parte da rotina profissional de quem trabalha com dados em ambientes distribuídos, garantindo que a fonte esteja disponível antes de iniciar qualquer processamento.

```
dbutils.fs.ls("/databricks-datasets/samples/population-vs-price")
```

Ao executar esse comando, o Databricks retornará a lista de arquivos presentes no diretório, incluindo o arquivo `data_geo.csv`. Caso o arquivo seja exibido, o ambiente está pronto para a execução do hands on. Essa etapa dispensa qualquer necessidade de upload manual de dados, tornando a atividade totalmente reprodutível mesmo em ambientes gratuitos.

Com o dataset validado, inicia-se a manipulação de dados utilizando **Apache Spark** por meio da API de DataFrames. Nesta etapa, você irá carregar o arquivo CSV, explorando suas colunas e compreendendo sua estrutura inicial, como parte do processo de análise exploratória.

Passo a Passo — Preparando o Ambiente

Passo 1 — Acessar o Databricks Free

1. Acesse: https://community.cloud.databricks.com.
2. Faça login ou crie uma conta gratuita.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

3. Após o login, você será direcionado ao **Workspace.**

Passo 2 — Criar um Notebook

- No menu lateral esquerdo, clique em **Workspace.**
- Clique com o botão direito → **Create** → **Notebook.**

Nomeie o notebook, por exemplo: `HandsOn_SparkAgg_Databricks.`

Selecione:

- Language: Python.
- Cluster: Starter (padrão do Community Edition).

Passo 3 — Validar o Ambiente

Execute a célula abaixo para validar o ambiente Spark:

python

```
spark
```

Se o objeto `SparkSession` for exibido, o ambiente está pronto.

Manipulação de Dados com Spark

Passo 4 — Validar a localização do arquivo de dados

- O Databricks disponibiliza datasets públicos nativos no **DBFS.**
- O arquivo utilizado neste hands on está disponível em: /databricks-datasets/samples/population-vs-price/data_geo.csv.
- Para validar a existência do arquivo, execute: dbutils.fs.ls("/databricks-datasets/samples/population-vs-price").
- Se o arquivo `data_geo.csv` aparecer na listagem, o ambiente está pronto para a leitura dos dados.

Passo 5 — Leitura do arquivo CSV

Realize a leitura do arquivo utilizando a API de DataFrames do Spark:

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
     df = (
         spark.read
              .option("header", "true")
              .option("inferSchema", "true")
     .csv("/databricks-datasets/samples/population-vs-
price/data_geo.csv")
     )

     df.show(5)
```

Código – Fonte 1: Leitura do arquivo - Python
Fonte: Elaborado pelo autor (2026)

O DataFrame passa a representar os dados em memória distribuída.

Passo 6 — Seleção e filtragem de dados

- Selecione apenas as colunas relevantes.
- Renomeie colunas quando necessário.
- Aplique filtros baseados em regras de negócio.

```
     from pyspark.sql.functions import col

     df_filtrado = (
         df.select("City", "State", "2014 Population estimate")
           .withColumnRenamed("2014    Population    estimate",
"pop_2014")
           .filter(col("pop_2014") > 500000)
     )

     df_filtrado.show()
```

Código – Fonte 2: Seleção e filtragem de dados - Python
Fonte: Elaborado pelo autor (2026)

Passo 7 — Criação de colunas derivadas

- Crie novas colunas a partir de cálculos sobre dados existentes.
- Essa prática é comum em processos analíticos e pipelines de dados.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
from pyspark.sql.functions import round

df_transformado = df_filtrado.withColumn(
    "pop_milhoes",
    round(col("pop_2014") / 1_000_000, 2)
)

df_transformado.show()
```

Código – Fonte 3: Criação de colunas - Python
Fonte: Elaborado pelo autor (2026)

Passo 8 — Agregações

- Utilize operações de agregação para resumir grandes volumes de dados.
- Agrupe os dados por estado e conte o número de cidades.

```
df_agrupado = (
    df_transformado
        .groupBy("State")
        .count()
        .orderBy(col("count").desc())
)

df_agrupado.show()
```

Código – Fonte 4: Agregações - Python
Fonte: Elaborado pelo autor (2026)

Passo 9 — Consultas com Spark SQL

- Crie uma view temporária a partir do DataFrame.
- Execute consultas utilizando linguagem SQL.

```
df_transformado.createOrReplaceTempView("cidades")

resultado_sql = spark.sql("""
    SELECT State, AVG(pop_2014) AS media_populacao
    FROM cidades
    GROUP BY State
    ORDER BY media_populacao DESC
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
""")

resultado_sql.show()
```

Código – Fonte 5: Consultas - Python
Fonte: Elaborado pelo autor (2026)

Orientação final ao aluno

- Pause o vídeo sempre que necessário.
- Reexecute os comandos no notebook.
- Experimente:
  - Alterar filtros;
  - Criar novas colunas;
  - Escrever consultas SQL diferentes.

Essa exploração prática é essencial para consolidar o entendimento sobre **manipulação de dados distribuídos com Apache Spark no Databricks**.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SAIBA MAIS

Manipulação de Dados com Spark

A **manipulação de dados com Spark** é um dos conhecimentos centrais para quem atua com dados em ambientes modernos. Nesta aula, você entrou em contato com operações fundamentais que aparecem diariamente em projetos de **engenharia de dados, analytics e ciência de dados**. Nesta seção, vamos retomar cada conceito com calma, explicando *o que é*, *para que serve* e *como usar*, sempre conectando teoria e prática com exemplos claros.

O motor por trás de tudo isso é o **Apache Spark**, utilizado amplamente em plataformas como o **Databricks**. O grande diferencial do Spark é permitir que você escreva código simples, enquanto o processamento acontece de forma distribuída e paralela.

O que significa manipular dados no Spark?

Manipular dados no Spark significa **ler dados, transformá-los e gerar novas informações**, tudo isso considerando que os dados podem ser muito grandes para caber em um único computador. Diferentemente de ferramentas tradicionais, o Spark distribui os dados em partes menores, chamadas de **partições**, que são processadas simultaneamente por vários nós do cluster.

Para o aluno, o mais importante é entender que:

- Você escreve o código como se estivesse lidando com uma tabela;
- O Spark cuida automaticamente da paralelização;
- As mesmas operações funcionam tanto para pequenos quanto para grandes volumes de dados.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SparkSession: o ponto de partida de tudo

Toda interação com o Spark começa a partir da **SparkSession**. Ela é o "ambiente de trabalho" da aplicação Spark e centraliza todas as operações.

```
spark
```

Quando esse comando retorna um objeto do tipo `SparkSession`, significa que:

- O cluster está ativo;
- O Spark está pronto para receber comandos;
- Você pode ler dados, criar DataFrames e executar SQL.

No Databricks, essa sessão já vem pronta, o que simplifica bastante o início do trabalho.

DataFrames: a principal estrutura de dados

O **DataFrame** é a estrutura mais importante do Spark. Ele representa dados organizados em linhas e colunas, de forma muito semelhante a uma tabela de banco de dados relacional.

A grande diferença é que:

- O DataFrame é **distribuído**;
- Os dados podem estar espalhados por vários nós;
- As operações são executadas em paralelo.

Quando você lê um arquivo CSV no Spark, o resultado é um DataFrame.

```
df = spark.read.csv("arquivo.csv")
```

A partir desse momento, todas as manipulações acontecem sobre esse objeto.

Leitura de dados: trazendo os dados para o Spark

No hands on, a leitura foi feita a partir de um arquivo CSV:

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
df = (
    spark.read
        .option("header", "true")
        .option("inferSchema", "true")
        .csv("/databricks-datasets/samples/population-vs-price/data_geo.csv")
)
```

Código – Fonte 6: CSV em Python
Fonte: Elaborado pelo autor (2026)

Vamos entender cada parte:

- `spark.read`: inicia o processo de leitura;
- `option("header", "true")`: indica que a primeira linha contém os nomes das colunas;
- `option("inferSchema", "true")`: pede para o Spark identificar automaticamente os tipos das colunas;
- `csv(...)`: define o formato e o caminho do arquivo.

Essa leitura já acontece de forma paralela, sem que você precise se preocupar com isso.

Ações e transformações: como o Spark executa o código

No Spark, nem tudo é executado imediatamente. Existem dois tipos principais de operações:

- **Transformações**: descrevem o que deve ser feito com os dados.
- **Ações**: disparam a execução real do processamento.

Por exemplo:

```
df.select("City", "State")
```

Isso é apenas uma transformação. O Spark ainda não processou nada.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Já o df.show(), é uma ação. Nesse momento, o Spark executa todas as transformações pendentes.

Esse comportamento é chamado de **execução lazy** e permite que o Spark otimize o processamento antes de executá-lo.

Visualizando dados com `show()`

A função `show()` é usada para visualizar os dados:

```
df.show(5)
```

Ela exibe as primeiras linhas do DataFrame e é muito útil para:

- Entender a estrutura dos dados;
- Validar se a leitura ocorreu corretamente;
- Apoiar a análise exploratória inicial.

Mesmo sendo simples, `show()` já força o Spark a executar o pipeline até aquele ponto.

Seleção de colunas: trabalhando apenas com o necessário

A função `select()` permite escolher apenas as colunas relevantes:

```
df.select("City", "State", "2014 Population estimate")
```

Essa prática é importante porque:

- Reduz o volume de dados processados;
- Melhora a performance;
- Deixa o código mais claro e organizado.

Em projetos reais, selecionar apenas o necessário é uma regra básica de boas práticas.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Renomeando colunas para facilitar o uso

Nomes de colunas longos ou pouco claros dificultam a leitura do código. Por isso, utilizamos:

`df.withColumnRenamed("2014 Population estimate", "pop_2014")`

Essa função cria um novo DataFrame com o nome da coluna ajustado, mantendo o original intacto. No Spark, os DataFrames são **imutáveis**, ou seja, cada transformação gera um novo objeto.

Filtragem de dados: aplicando regras de negócio

A filtragem é feita com `filter()` ou `where()`:

```
from pyspark.sql.functions import col

df.filter(col("pop_2014") > 500000)
```

Aqui estamos aplicando uma regra de negócio simples: considerar apenas cidades com população maior que 500 mil habitantes.

Filtros bem aplicados:

- Reduzem o volume de dados;
- Melhoram a performance;
- Deixam o pipeline mais eficiente.

Criação de colunas derivadas: enriquecendo os dados

A função `withColumn()` permite criar novas colunas:

```
from pyspark.sql.functions import round

df.withColumn(
    "pop_milhoes",
    round(col("pop_2014") / 1_000_000, 2)
)
```

Código – Fonte 7: Código PySpark (Python)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Fonte: Elaborado pelo autor (2026)

Esse tipo de operação é chamado de **enriquecimento de dados**. Ele transforma dados brutos em informações mais úteis para análise, relatórios e indicadores.

Lógica condicional com `when` e `otherwise`

O Spark também permite aplicar regras condicionais:

```
from pyspark.sql.functions import when

df.withColumn(
    "categoria_pop",
    when(col("pop_2014") > 1_000_000, "Grande")
    .otherwise("Media")
)
```

Código – Fonte 8: código PySpark (Python)
Fonte: Elaborado pelo autor (2026)

Essa abordagem é muito utilizada para:

- Segmentações;
- Classificações;
- Aplicação de regras de negócio.

Agregações: resumindo grandes volumes de dados

As agregações transformam dados detalhados em informações consolidadas:

```
df.groupBy("State").count()
```

Outros exemplos comuns incluem:

```
df.groupBy("State").avg("pop_2014")

df.groupBy("State").max("pop_2014")

df.groupBy("State").min("pop_2014")
```

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Essas operações são mais custosas, pois envolvem redistribuição de dados entre os nós do cluster, mas são essenciais para análises e relatórios.

Ordenação e limitação de resultados

Para ordenar resultados, utilizamos `orderBy()`:

```
df.orderBy(col("count").desc())
```

Para limitar a quantidade de registros:

```
df.limit(10)
```

Essas funções são muito usadas em rankings e análises exploratórias.

Spark SQL: usando SQL sobre dados distribuídos

Ao criar uma view temporária:

```
df.createOrReplaceTempView("cidades"),
```

é possível consultar os dados usando SQL:

```
spark.sql("""
    SELECT State, AVG(pop_2014) AS media_pop
    FROM cidades
    GROUP BY State
""")
```

Código – Fonte 9: SQL (Structured Query Language) dentro do ambiente PySpark
Fonte: Elaborado pelo autor (2026)

Isso facilita o uso do Spark por profissionais que já dominam SQL, promovendo integração entre diferentes perfis de dados.

Cache e persistência: melhorando a performance

Quando um DataFrame é reutilizado várias vezes, pode ser interessante mantê-lo em memória:

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
df.cache()
```

Isso evita recomputações e melhora a performance em pipelines mais longos.

Fechando o aprendizado

As operações apresentadas nesta aula representam o **núcleo da manipulação de dados com Spark**. Elas aparecem repetidamente em projetos reais e formam a base para conteúdos mais avançados, como Delta Lake, otimização de performance e pipelines de dados em larga escala.

Dica final: volte ao notebook, execute cada célula com calma e experimente pequenas variações. A prática é o caminho mais eficaz para transformar conceitos em domínio real.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

MERCADO, CASES E TENDÊNCIAS

A **manipulação de dados com Apache Spark** consolidou-se como uma prática essencial no mercado de dados, especialmente em organizações que lidam com grandes volumes de informação e precisam transformar dados brutos em dados prontos para análise. O Spark é amplamente utilizado como motor central para operações de leitura, transformação, enriquecimento e agregação de dados, servindo como base para camadas analíticas, produtos digitais e aplicações de Machine Learning.

No contexto corporativo, o Spark é adotado como componente estratégico de pipelines de dados, permitindo que equipes de engenharia e analytics implementem fluxos escaláveis e confiáveis de manipulação de dados. Sua API de alto nível baseada em DataFrames e SQL reduz a complexidade do desenvolvimento, ao mesmo tempo em que mantém o poder do processamento distribuído, característica fundamental em ambientes de Big Data.

Grande parte das práticas aplicadas pelo mercado tem como base o conhecimento disseminado por publicações da O'Reilly, amplamente reconhecidas como referência em engenharia de dados e sistemas distribuídos. O livro **Learning Spark: Lightning-Fast Data Analytics** (Jules S. Damji, Brooke Wenig, Tathagata Das e Denny Lee) é uma das principais fontes para profissionais que atuam com Spark, pois apresenta de forma prática como realizar operações de manipulação de dados utilizando DataFrames, Spark SQL, transformações e agregações em ambientes reais de produção.

Outra obra de referência é **Spark: The Definitive Guide**, (Bill Chambers e eMatei Zaharia) que aprofunda o funcionamento interno do Spark e explica como conceitos como execução *lazy*, DAGs e otimizações automáticas permitem que operações de manipulação de dados escalem para grandes volumes, mantendo desempenho e confiabilidade.

Em cenários corporativos mais maduros, o desempenho dos pipelines torna-se uma preocupação central. Nesse contexto, o livro **High Performance Spark** (Holden Karau, Adi Polak e Rachel Warren) aborda boas práticas para otimização de pipelines de dados, discutindo uso de cache, particionamento e escolha adequada de

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

transformações e ações. Essas práticas refletem desafios reais enfrentados por equipes que manipulam dados em escala diariamente.

Complementando o embasamento técnico, a obra **Designing Data-Intensive Applications**, de Martin Kleppmann, é amplamente utilizada no mercado para fundamentar decisões arquiteturais. Embora não seja exclusiva de Spark, ela fornece a base conceitual sobre sistemas distribuídos, pipelines de dados, escalabilidade e tolerância a falhas, que sustenta o uso do Spark em arquiteturas modernas.

Além dos livros, artigos técnicos ajudam a reforçar conceitos abordados em sala. Um exemplo é o artigo **“What Is Apache Spark?”**, publicado pela O’Reilly, que apresenta uma visão clara e didática sobre o papel do Spark no processamento distribuído, seus principais componentes e por que ele se tornou uma ferramenta central para manipulação de dados em larga escala.

Conteúdos em formato de áudio também têm ganhado espaço como fonte de atualização profissional. O **Data Engineering Podcast**, mantido por profissionais da área e hospedado pela O’Reilly, apresenta episódios dedicados a Apache Spark, engenharia de dados e arquiteturas modernas, discutindo desafios reais de manipulação e transformação de dados em ambientes produtivos.

Como tendência, observa-se a consolidação do **Apache Spark como engine padrão para manipulação de dados distribuídos**. Cada vez mais, o Spark é utilizado como etapa central de preparação de dados, conectando ingestão, transformação e consumo analítico. A integração com Spark SQL, arquiteturas Lakehouse e técnicas de Machine Learning reforça a importância do domínio dessas operações para profissionais que atuam em projetos modernos de dados.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você compreendeu como a **manipulação de dados com Apache Spark** se torna um elemento central no processamento distribuído, permitindo trabalhar com grandes volumes de dados de forma escalável, eficiente e com uma sintaxe de alto nível. Foi possível entender como o Spark abstrai a complexidade da distribuição dos dados, possibilitando que operações comuns de análise sejam executadas em paralelo sobre um cluster.

Ao longo do conteúdo, você explorou o conceito de **DataFrames distribuídos**, entendendo como dados estruturados podem ser lidos, transformados e enriquecidos por meio de operações como seleção de colunas, filtragem, criação de colunas derivadas e agregações. Também foi discutido o modelo de execução *lazy* do Spark, no qual transformações são otimizadas e executadas apenas quando uma ação é solicitada.

A aula também destacou a importância das **funções da API do Spark**, como `select`, `filter`, `withColumn`, `groupBy` e `orderBy`, mostrando como elas são utilizadas para aplicar regras de negócio, resumir informações e preparar dados para análises. Além disso, você conheceu a integração entre DataFrames e **Spark SQL**, que permite consultar dados distribuídos utilizando linguagem SQL, facilitando o trabalho colaborativo entre engenheiros e analistas de dados.

Por fim, o hands on no **Databricks** consolidou o aprendizado ao demonstrar, na prática, um fluxo completo de manipulação de dados: desde a validação do ambiente, leitura de arquivos no DBFS, aplicação de transformações e agregações, até a execução de consultas SQL. Esse exercício reforçou a visão prática de como o Spark é utilizado em arquiteturas modernas de dados para transformar dados brutos em informações prontas para análise.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

REFERÊNCIAS

CHAMBERS, B.; ZAHARIA, M. **Spark: The Definitive Guide**. Sebastopol: O'Reilly Media, 2018.

DATA ENGINEERING PODCAST. **Apache Spark and Modern Data Engineering**. Disponível em: https://www.dataengineeringpodcast.com/. Acesso em: 26 fev. 2026.

KARAU, H.; WARREN, R. **High Performance Spark**. Sebastopol: O'Reilly Media, 2017.

KLEPPMANN, M. **Designing Data-Intensive Applications**. Sebastopol: O'Reilly Media, 2017.

O'REILLY MEDIA. **What Is Apache Spark?** Radar O'Reilly. Disponível em: https://www.oreilly.com/radar/what-is-apache-spark/. Acesso em: 26 fev. 2026.

ZAHARIA, M.; CHAMBERS, B.; WENIG, B.; DAMJI, J. **Learning Spark: Lightning-Fast Data Analytics**. Sebastopol: O'Reilly Media, 2020.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

PALAVRAS-CHAVE

Apache Spark. Processamento Distribuído. Manipulação de Dados.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com



PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com